$s225269_Lecture_4$

Jonas Pedersen

February 2026

1 Exercise 1 – USART Transmit

1.1 Setup

USART2 was configured in CubeMX in asynchronous mode with the following settings:

- Baud rate: 9600
- Word length: 8 bit
- Stop bits: 1
- Parity: None
- Mode: TX/RX

1.2 Clock Configuration

The following clock speeds were configured in CubeMX:

- HCLK: 64 MHz
- APB1: 32 MHz (HCLK divided by 2)
- APB2: 64 MHz (no division)

According to the STM32F302 reference manual, USART2 is clocked from the **APB1** bus. The clock frequency relevant for USART2 is therefore **32 MHz**. This is the frequency used internally by the USART2 baud rate generator to derive the 9600 baud rate.

1.3 usart.c – Initialization

CubeMX generates a `usart.c` file with two central functions. `MX_USART2_UART_Init()` configures the UART protocol by populating a `UART_HandleTypeDef` struct (`huart2`) with the chosen parameters and then calls `HAL_UART_Init()`, which writes the settings to the hardware registers.

`HAL_UART_MspInit()` handles the low-level hardware setup (MSP = MCU Support Package). It enables the clock for the USART2 peripheral and GPIOA, and configures PA2 (TX) and PA3 (RX) as Alternate Function pins with AF7, internally connecting them to USART2 inside the chip.

1.4 HAL_UART_Transmit

According to the STM32 HAL manual, `HAL_UART_Transmit()` has the following signature:

```
HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *
    huart,
                                        const uint8_t *pData,
                                        uint16_t Size,
                                        uint32_t Timeout);
```

The parameters are as follows:

- `huart` – pointer to the UART handle struct, in our case `&huart2`. This tells the function which UART peripheral to use.
- `pData` – pointer to the data buffer to transmit, cast to `uint8_t*` since UART works on raw bytes.
- `Size` – number of bytes to transmit. We use `strlen()` or `snprintf()` to determine the correct length.
- `Timeout` – maximum time in milliseconds to wait for transmission to complete. `HAL_MAX_DELAY` means wait indefinitely.

The function returns `HAL_OK` if transmission was successful, `HAL_TIMEOUT` if it timed out, or `HAL_ERROR` if something went wrong.

1.5 Code

1.5.1 usart.c

Listing 1: usart.c

```
void MX_USART2_UART_Init(void)
{

  /* USER CODE BEGIN USART2_Init 0 */

  /* USER CODE END USART2_Init 0 */

  /* USER CODE BEGIN USART2_Init 1 */

  /* USER CODE END USART2_Init 1 */
  huart2.Instance = USART2;
```

```
  huart2.Init.BaudRate = 9600;
  huart2.Init.WordLength = UART_WORDLENGTH_8B;
  huart2.Init.StopBits = UART_STOPBITS_1;
  huart2.Init.Parity = UART_PARITY_NONE;
  huart2.Init.Mode = UART_MODE_TX_RX;
  huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart2.Init.OverSampling = UART_OVERSAMPLING_16;
  huart2.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
  huart2.AdvancedInit.AdvFeatureInit =
      UART_ADVFEATURE_NO_INIT;
  if (HAL_UART_Init(&huart2) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN USART2_Init 2 */

  /* USER CODE END USART2_Init 2 */

}
```

1.5.2 main.c

Listing 2: main.c

```
#include "main.h"
#include "usart.h"
#include "gpio.h"
#include <stdio.h>
#include "usart.h"
MX_GPIO_Init();
  MX_USART2_UART_Init();
  /* USER CODE BEGIN 2 */

  /* USER CODE END 2 */

  /* USER CODE BEGIN 2 */
  char msg[50];
  int counter = 0;
  /* USER CODE END 2 */

  /* USER CODE BEGIN WHILE */
  while (1)
  {
      int len = snprintf(msg, sizeof(msg), "Counter␣=␣%d\r\n
          ", counter);
      HAL_UART_Transmit(&huart2, (uint8_t*)msg, len,
          HAL_MAX_DELAY);
      counter++;
      HAL_Delay(1000);
```

```
      /* USER CODE END WHILE */

      /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
  /* USER CODE END 3 */
}
```

1.6 Result

As shown in Figure 1, messages are correctly received in the terminal on the mac, with the counter incrementing by 1 every second.

Figure 1: Terminal output from USART2 on Mac

1.7 Questions

Q1: Difference between USART and UART
UART only supports asynchronous communication, where timing is controlled via baud rate with no shared clock signal. USART supports both asynchronous and synchronous modes, where a CLK signal is used to synchronize data between transmitter and receiver.

Q2: Pros and cons of USART vs UART
USART is more flexible and can operate faster in synchronous mode, but requires an additional CLK line. UART is simpler and only requires TX/RX lines, but depends on precise baud rate matching on both sides.

Q3: Number of USART/UARTs on F302
The STM32F302R8 has 3 USARTs: USART1, USART2 and USART3.

Q4: USART2 TX/RX on morpho connector
USART2 TX = PA2 and RX = PA3. On the Nucleo-64 morpho connector these are located on CN10 (PA2 = pin 35, PA3 = pin 37).

Q5: GPIOs used by USART3
USART3 TX = PB10 and RX = PB11.

Q6: Why USART2 rather than USART3?
USART2's TX/RX pins (PA2/PA3) are hardwired to the ST-LINK VCP (Virtual COM Port) on the Nucleo board, meaning data is automatically forwarded over USB to the PC without any additional cables. USART3 has no direct connection to the ST-LINK.

Q7: Debugging USART3 on PC terminal
An external USB-to-UART adapter (e.g. FTDI or CP2102) would need to be connected to USART3's TX/RX pins, and the corresponding port opened in a terminal program on the PC.

Q8: Minimum HCLK when USARTs are enabled
The minimum acceptable HCLK is 8 MHz. Below this, the USART baud rate generator cannot produce standard baud rates with sufficient precision.

Q9: What happens if APB1 is set to 4 MHz?
USART2 and USART3 are clocked from the APB1 bus. At 4 MHz, the baud rate generator cannot correctly produce standard baud rates, causing communication to fail with corrupted or no data being received.

Q10: Why is USART1 not affected by reducing APB1?
USART1 is connected to the APB2 bus, not APB1. APB2 runs independently, so USART1 remains unaffected when APB1 is reduced.

Q11: Did we need the Nucleo RX to TX cable connection?
No. Since we are only transmitting from the MCU to the PC (one-way), only the TX line is needed. The ST-LINK VCP handles this connection automatically over USB.

2 Exercise 2 – USART Receive with Interrupt

2.1 Setup

Exercise 2 builds on the same USART2 configuration as Exercise 1, but adds interrupt-based reception. In CubeMX, the USART2 global interrupt was enabled under **Connectivity → USART2 → NVIC Settings**. Without this, the interrupt callback will never be triggered.

2.2 How the Code Works

The program accepts alphanumeric characters from a terminal program and echoes them back. It also detects a specific character sequence ("LD2 on") and lights the user LED once it is detected.

Because any received character triggers an interrupt, it is critical that the USART2 global interrupt is enabled. This is configured in CubeMX under the

NVIC settings for USART2, which causes the HAL to automatically generate the associated callback function `HAL_UART_RxCpltCallback()`, where all interrupt handling takes place.

2.3 Interrupt Principle

The interrupt-based approach works as follows:

1. At startup, `HAL_UART_Receive_IT()` is called once to arm the interrupt and tell the UART to listen for 1 incoming byte
2. When a character arrives on USART2, a hardware interrupt fires and `HAL_UART_RxCpltCallback()` is called automatically by the HAL
3. Inside the callback, the received character is added to `Rx_Buffer`
4. Each character is echoed back to the terminal using `HAL_UART_Transmit()`, so the user can see what they are typing
5. The interrupt is immediately re-armed with another call to `HAL_UART_Receive_IT()` to listen for the next character
6. When a carriage return (`\r`) is received, signalling that the user pressed Enter, the `Transfer_cplt` flag is set to 1
7. In the main loop, when `Transfer_cplt` is 1, the buffer is compared to "LD2 on" using `strcmp()`. If they match, the LED on PB13 is turned on
8. The buffer is cleared and the flag is reset, ready for the next input

The key advantage of this approach is that the CPU is **not blocked** waiting for input. Between received characters, the main loop runs freely and could perform other tasks. The interrupt only briefly takes over when a character actually arrives.

2.4 Code

2.4.1 main.c

Listing 3: main.c – Exercise 2

```c
#include "main.h"
#include "usart.h"
#include "gpio.h"

/* USER CODE BEGIN Includes */
#include <string.h>
/* USER CODE END Includes */

/* USER CODE BEGIN PV */
```

```c
int Rx_indx = 0;
char Rx_data[2];
char Rx_Buffer[100];
char Transfer_cplt = 0;
/* USER CODE END PV */

void SystemClock_Config(void);

int main(void)
{
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  MX_USART2_UART_Init();

  /* USER CODE BEGIN 2 */
  HAL_UART_Receive_IT(&huart2, (uint8_t*)Rx_data, 1);
  /* USER CODE END 2 */

  /* USER CODE BEGIN WHILE */
  while (1)
  {
      if (Transfer_cplt == 1)
      {
          if (strcmp(Rx_Buffer, "LD2 on") == 0)
          {
                  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_13, GPIO_PIN_SET);
          }
          memset(Rx_Buffer, 0, sizeof(Rx_Buffer));
          Rx_indx = 0;
          Transfer_cplt = 0;
          HAL_UART_Receive_IT(&huart2, (uint8_t*)Rx_data, 1);
      }
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}

/* USER CODE BEGIN 4 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2)
    {
        Rx_Buffer[Rx_indx++] = Rx_data[0];

        if (Rx_data[0] == '\r')
```

```
        {
            Rx_Buffer[Rx_indx - 1] = '\0';
            Transfer_cplt = 1;
        }
        else
        {
            HAL_UART_Transmit(&huart2, (uint8_t*)Rx_data, 1,
                 HAL_MAX_DELAY);
            HAL_UART_Receive_IT(&huart2, (uint8_t*)Rx_data,
                1);
        }
    }
}
/* USER CODE END 4 */
```

2.5 Result

When the string "LD2 on" is typed in the terminal followed by Enter, the green LED (LD2) on PB13 turns on. Each character is echoed back to the terminal as it is typed, confirming that both reception and transmission work correctly over USART2.

3 Exercise 3 – USART Receive and Transmit with Polling

3.1 Setup

Exercise 3 uses the same USART2 configuration as the previous exercises. Unlike Exercise 2, no interrupt is needed as the program uses polling mode, where the CPU actively waits for incoming data using `HAL_UART_Receive()`.

3.2 Polling Principle

In polling mode, the CPU is blocked inside `HAL_UART_Receive()` until either data is received or a timeout occurs. This means the program cannot do anything else while waiting for input. Although this is less efficient than interrupt-based reception, it is simpler to implement and sufficient for applications where no other tasks need to run concurrently.

The program works as follows:

1. `HAL_UART_Receive()` is called with a timeout of 5000 ms, listening for 1 byte at a time
2. Each received character is added to an internal buffer
3. When a carriage return (`\r`) or newline (`\n`) is received, the buffer is compared to the known commands

4. If the buffer matches `"LED_ON"`, the green LED on PB13 is turned on and `"LED is ON"` is transmitted back
5. If the buffer matches `"LED_OFF"`, the LED is turned off and `"LED is OFF"` is transmitted back
6. If the input does not match any known command, `"ERROR"` is returned
7. If nothing is received within 5 seconds, `"TIMEOUT"` is returned
8. The buffer is cleared and the process repeats

3.3 Code

3.3.1 main.c

Listing 4: main.c – Exercise 3

```c
#include "main.h"
#include "usart.h"
#include "gpio.h"

/* USER CODE BEGIN Includes */
#include <string.h>
#include <stdio.h>
/* USER CODE END Includes */

/* USER CODE BEGIN PV */
char Rx_Buffer[10];
/* USER CODE END PV */

void SystemClock_Config(void);

int main(void)
{
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  MX_USART2_UART_Init();

  /* USER CODE BEGIN WHILE */
  while (1)
  {

      HAL_StatusTypeDef status = HAL_UART_Receive(&huart2,
                                                  (uint8_t
                                                      *)
                                                      Rx_Buffer
                                                      ,
                                                  1,
```

```
                                                    5000);
        if (status == HAL_TIMEOUT)
        {
            char *timeout_msg = "TIMEOUT\r\n";
            HAL_UART_Transmit(&huart2, (uint8_t*)timeout_msg,
                              strlen(timeout_msg),
                                  HAL_MAX_DELAY);
        }
        else if (status == HAL_OK)
        {
            static int idx = 0;
            static char buf[20];

            if (Rx_Buffer[0] == '\r' || Rx_Buffer[0] == '\n')
            {
                buf[idx] = '\0'; // afslut string

                if (strcmp(buf, "LED_ON") == 0)
                {
                    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_13,
                        GPIO_PIN_SET);
                    char *on_msg = "LED␣is␣ON\r\n";
                    HAL_UART_Transmit(&huart2, (uint8_t*)
                        on_msg,
                                      strlen(on_msg),
                                          HAL_MAX_DELAY);
                }
                else if (strcmp(buf, "LED_OFF") == 0)
                {
                    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_13,
                        GPIO_PIN_RESET);
                    char *off_msg = "LED␣is␣OFF\r\n";
                    HAL_UART_Transmit(&huart2, (uint8_t*)
                        off_msg,
                                      strlen(off_msg),
                                          HAL_MAX_DELAY);
                }
                else
                {
                    char *error_msg = "ERROR\r\n";
                    HAL_UART_Transmit(&huart2, (uint8_t*)
                        error_msg,
                                      strlen(error_msg),
                                          HAL_MAX_DELAY);
                }

                // Nulstil
                idx = 0;
                memset(buf, 0, sizeof(buf));
            }
```

```
            else
            {
                buf[idx++] = Rx_Buffer[0];
            }
        }
        /* USER CODE END WHILE */

        /* USER CODE BEGIN 3 */
    }
    /* USER CODE END 3 */
}
```

3.4 Result

As shown in the terminal output below, the program correctly responds to `LED_ON` and `LED_OFF` commands, returns `ERROR` for unknown input, and `TIMEOUT` when nothing is received within 5 seconds.

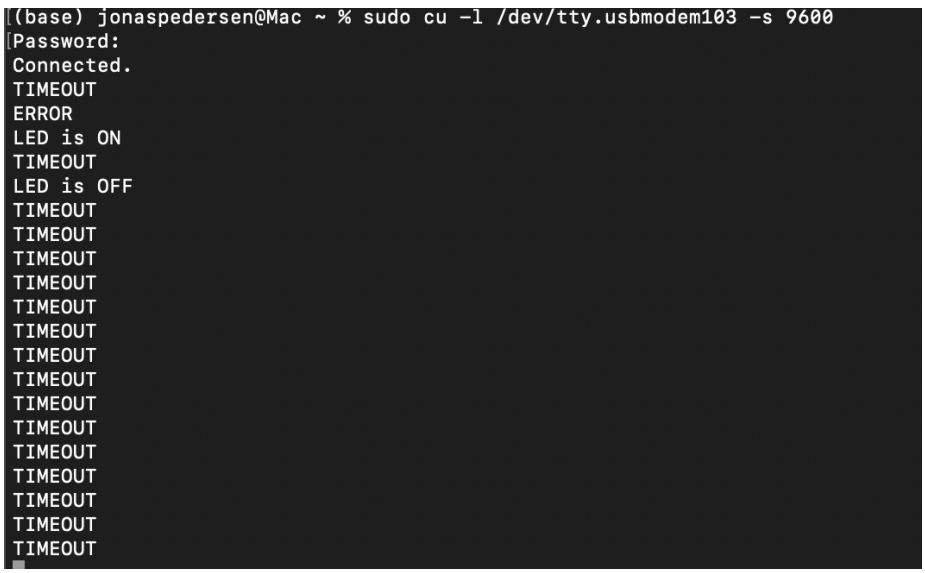

Figure 2: Terminal output from Exercise 3

3.5 Question – Interrupt vs Polling

The fundamental difference between Exercise 2 (interrupt) and Exercise 3 (polling) is how the MCU handles waiting for incoming data.

In **polling mode** (Exercise 3), the CPU is blocked inside `HAL_UART_Receive()` and cannot perform any other tasks while waiting. This wastes CPU cycles and makes the system unresponsive during the wait.

In **interrupt mode** (Exercise 2), the CPU runs freely and is only briefly interrupted when a character actually arrives. This is far more efficient as the CPU can handle other tasks between received characters.

For most embedded systems, interrupt-based reception is therefore preferred, especially when the MCU needs to handle multiple peripherals or tasks simultaneously. Polling is only practical for very simple applications where no other processing is needed.